UWE Bristol University of the West of England

Lecture 6: OO Analysis and Design

Design Patterns

Week 6

Last week recap

- We covered
  - State Machine and Activity Diagrams
  - SOLID Principles (five principles)

- Practical
  - Activity Diagrams and a new case study (UC/CD/SD)

Outline

- Design Patterns
- Why Design Patterns?
- Different types and examples

OO Design

UWE Bristol University of the West of England

Is designing object-oriented software easy/hard?

OO Design

- Designing software for reuse is hard
- **SOLID principles**
  - A good problem decomposition, a design with flexibility and modularity

- **Code re-use**
  - Don’t reinvent the wheel (leverage previous work)
  - Requires clean, elegant, understandable, general, stable code

- **Design re-use**
  - Don’t reinvent the wheel (leverage previous work)
  - Requires precise understanding of common recurring designs

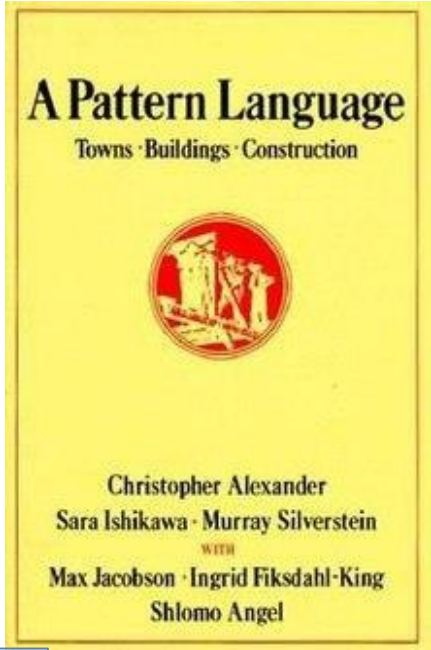
Pattern

- So what is a pattern?
- Where do patterns exist?
- Did you encounter repeating problems when you're learning how to program/write code?

History of Design Patterns

- Design patterns have their roots in the work of **Christopher Alexander**, a civil engineer (academic).

- The book creates a new language, what the authors call a pattern language derived from timeless entities called patterns. As they write on page xxxv of the introduction, "**All 253 patterns together form a language." Patterns describe a problem and then offer a solution**. In doing so the authors intend to give ordinary people, not only professionals, a way *to work with their neighbours to improve a town or neighbourhood, design a house for themselves or work with colleagues to design an office, workshop or public building such as a school*.- Wikipedia



"Each pattern describes a problem which occurs over and over again in our environment, and then describes the core of the solution to that problem, in such a way that you can use this solution a million times over, without ever doing it the same way twice."

Christopher Alexander, Architect

Design Patterns

UWE Bristol University of the West of England

- Each pattern is **a four-part rule**, which expresses a relation between a certain context, a problem, a solution, and consequences

- ***Design Patterns -- Elements of Reusable Object-oriented Software***, the GoF (gang of four: Erich, Richard, Ralph, John) defines design patterns as "descriptions of communicating objects and classes that are customized to solve a general design problem in a particular context."

- A design pattern is a **general reusable solution** to a commonly **occurring problem** within a given context in software design.

- A design pattern is **not a finished design** that can be transformed directly into source code.

- It is a **description or template for how to solve a problem** that can be used in many different situations.

Why use Design Patterns?

UWE Bristol University of the West of England

Specifically – design patterns solve design problems:

- **Find appropriate objects** – help you identify less-obvious abstractions and objects that capture them
- **Encourage reusability** – don't have to think about best approach
- **Determine object granularity** – patterns can help simplify the process of composition
- **Specify object interfaces** – help harden your interfaces, what's included and what's not included in the interface
- **Describe object implementations** – help you understand the implications of certain implementations
- **Support extensibility** – change and flexibility built right in

Types of Design Patterns (GoF Patterns)

UWE Bristol University of the West of England

- As per the design pattern reference book **Design Patterns - Elements of Reusable Object-Oriented Software**, there are 23 design patterns which can be classified in three categories:

Pattern	Description
Creational Pattern(s)	**Talks about how to create new objects** These design patterns provide a way to create objects.
Structural Pattern(s)	**Talks about composition of objects and classes** These design patterns concern class and object composition. They describe how classes and objects can be combined to form larger structures. Simplifies the structure by identifying the relationships
Behavioral Pattern(s)	**Talks about ways in which objects interact** These design patterns are specifically concerned with communication/interaction between objects. Also, provide loose coupling and flexibility to extend easily

Types of Design Patterns (23 GoF Patterns)

UWE Bristol University of the West of England

- **Creational Patterns**
  - Abstract Factory
  - Builder
  - Factory Method
  - Prototype
  - Singleton
- **Structural Patterns**
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - Façade
  - Flyweight
  - Proxy
- **Behavioral Patterns**
  - Chain of Responsibility
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - Observer
  - State
  - Strategy
  - Template Method
  - Visitor

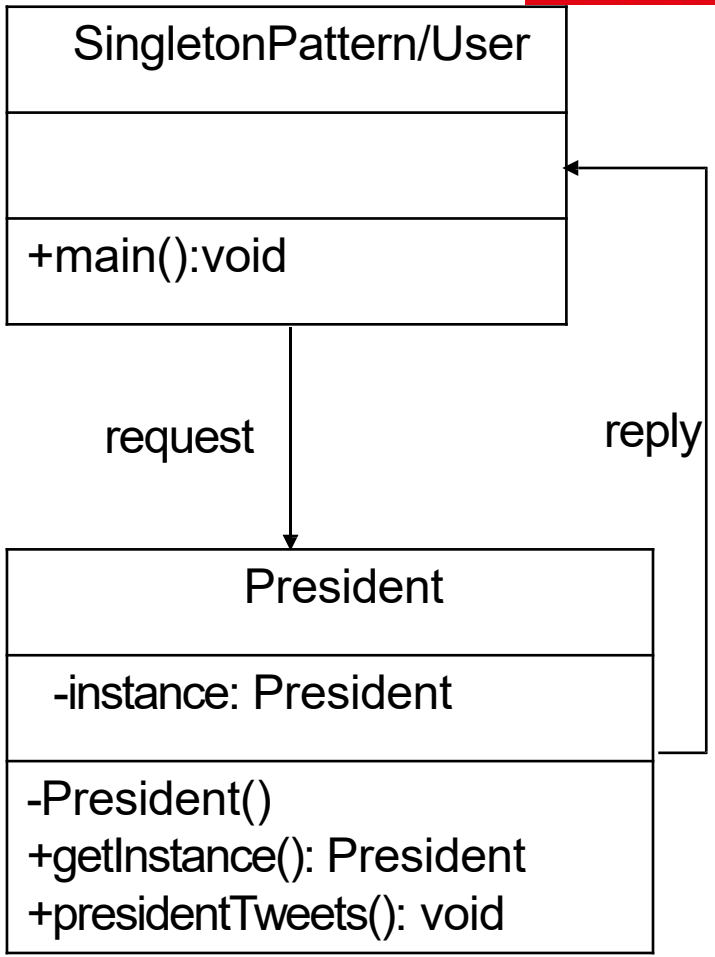
Types of Design Patterns (23 GoF Patterns)

UWE Bristol University of the West of England

- **Creational Patterns**
  - Abstract Factory
  - Builder
  - Factory Method
  - Prototype
  - Singleton
- **Structural Patterns**
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - Façade
  - Flyweight
  - Proxy
- **Behavioral Patterns**
  - Chain of Responsibility
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - Observer
  - State
  - Strategy
  - Template Method
  - Visitor

Singleton Pattern (GoF Patterns)

- **Intent:** Ensure a class has only one instance, and provide a global point of access to it.
- **Problem:** Application needs one, and only one, instance of an object. Additionally, global access is necessary.
- **Example:** 1. One cannot elect two presidents, the presidency has only one instance.
- **Example:** 2. creating single DB connection and reusing it to avoid unnecessary connections from multiple locations



Singleton Pattern

```
class President:

        #Create a President variable
        __president_instance = None

        def __init__(self):
                """virtual private constructor"""
                if President.__president_instance != None:
                        raise Exception("There can be only one President !")
                else:
                        President.__president_instance = self

        @staticmethod
        def getInstance():
                """Static Access Method"""
                if President.__president_instance == None:
                        President.__president_instance = President()
                return President.__president_instance

        def presidentTweets(self):  #additional methods/responsibilities
                print('Hello World! I will build a great wall - and nobody builds
                walls better than me, believe me.')
```

Singleton Pattern

```
# main can be in a separate SingletonPattern class.
if __name__ == "__main__":
        # create object of Singleton Class
        obj = President()
        print(obj)

        # pick the instance of the class
        obj1 = President.getInstance()
        print(obj1)

        # tweet method
        obj.presidentTweets()
        obj1.presidentTweets()
```

```
obj2 = President()
print(obj2)
raise Exception("There can be only one President !")
```

```
<__main__.President object at 0x103077e50>
<__main__.President object at 0x103077e50>
Hello World! I will build a great wall - and nobody builds walls better than me, believe me.
Hello World! I will build a great wall - and nobody builds walls better than me, believe me.
```

Singleton Pattern

- Eager over Lazyinitialization
- More favourable, when we are sure that we need only one instance

```
class President:

        #Create a President instance. Eager initialisation
        __president_instance = President()

        def __init__(self):
                """virtual private constructor"""
                if President.__president_instance != None:
                        raise Exception("There can be only one President !")
                else:
                        President.__president_instance = self

        #Get the only object available.
        @staticmethod
        def getInstance():
                """Static Access Method"""
                return President.__president_instance

        def presidentTweets(self): #additional methods/responsibilities
                print('Hello World! I will build a great wall - and nobody builds
                walls better than me, believe me.')
```

Singleton Pattern (GoF Patterns)

UWE Bristol University of the West of England

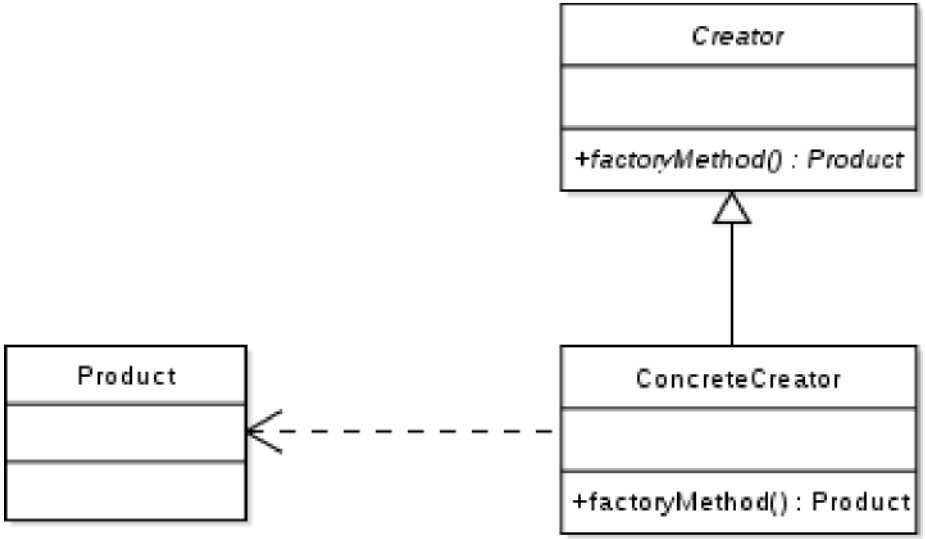
Homework

Do you see the need for Singleton Pattern in your group project?

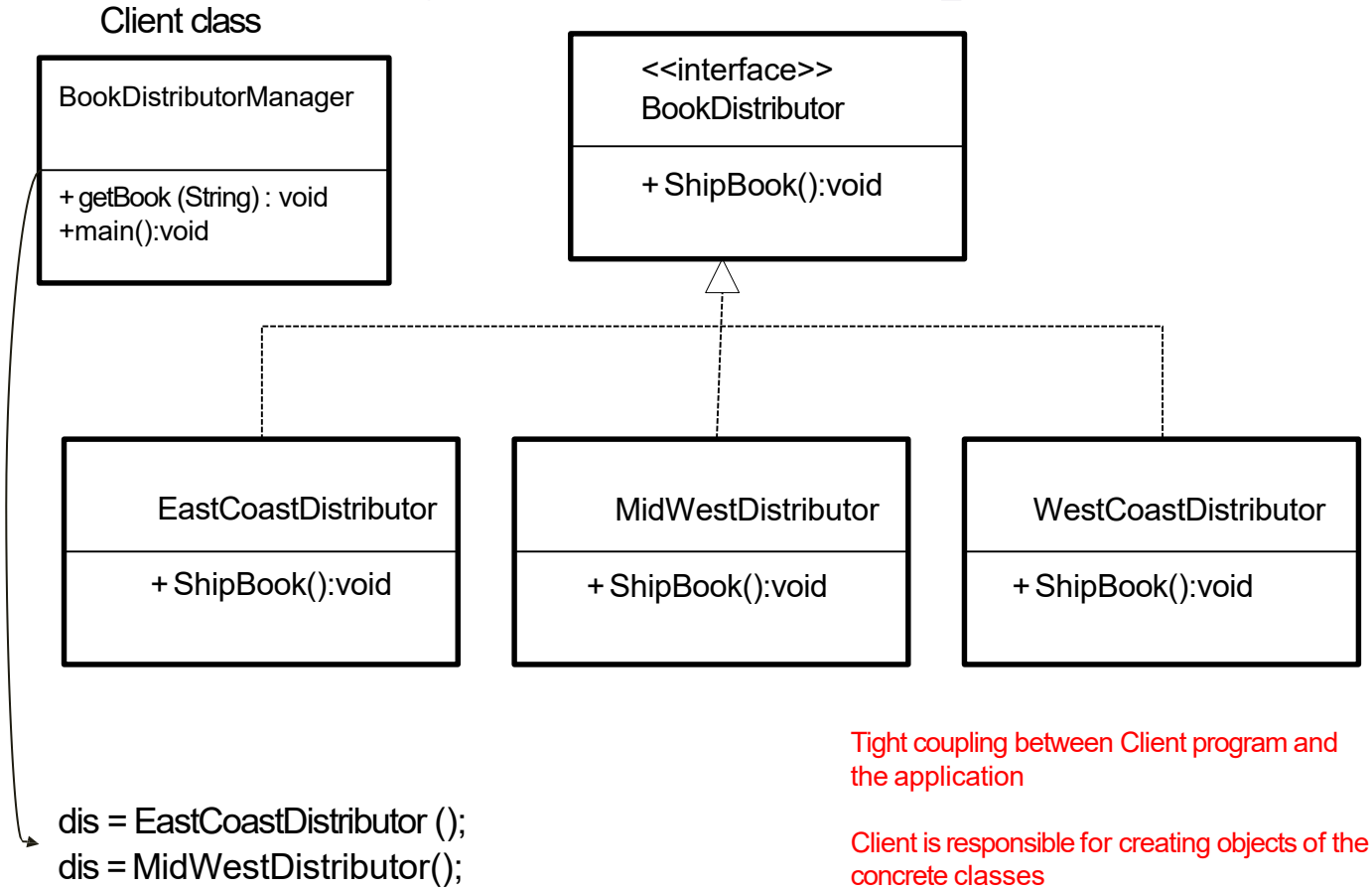
Come prepared for your next practical session

Factory Pattern

- Implements the concept of factories and deals with problem of creating objects without specifying exact class
- Define an interface for creating object, but let the classes that implement the interface decide which class to instantiate
- The Factory method lets a class defer instantiation to subclasses



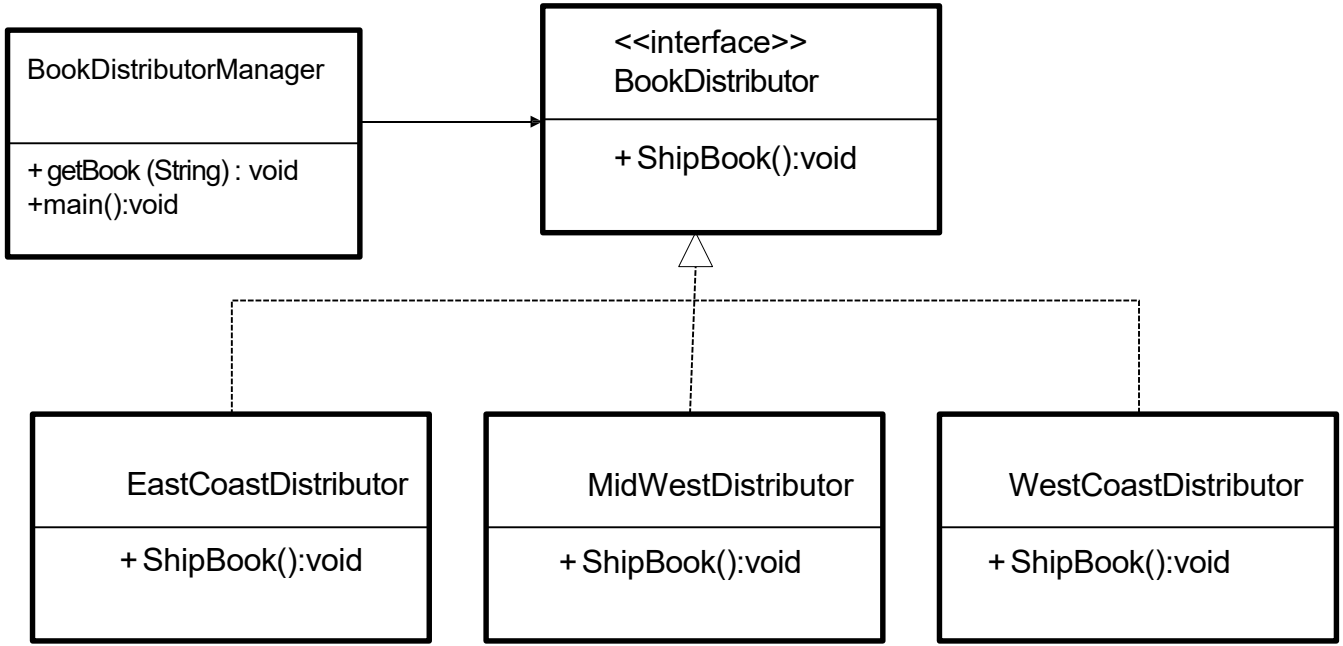
Factory Method Example



Tight coupling between Client program and the application

Client is responsible for creating objects of the concrete classes

Factory Method Example



- System has an Interface **BookDistributor** which defines the common behavior for all types of distributors
- All the product classes (namely EastCoastDistributor, MidWestDistributor, and WestCoastDistributor) implement this interface

Factory Method Example

```
from abc import ABCMeta, abstractmethod

# the product
class BookDistributor(metaclass = ABCMeta):
        @abstractmethod
        def ship_book():
                pass

# concrete product
class EastCoastDistributor(BookDistributor):
        def ship_book(self):
                print('Book shipped from East Coast')

# concrete product
class WestCoastDistributor(BookDistributor):
        def ship_book(self):
                print('Book shipped from West Coast')

# concrete product
class MidWestCoastDistributor(BookDistributor):
        def ship_book(self):
                print('Book shipped from MidWest Coast')
```

As per the user request, client application will instantiate the desired class

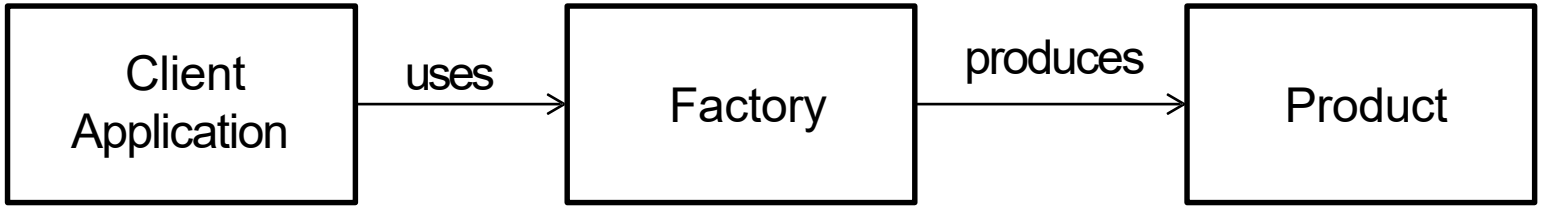
```
class BookDistributorManager:
        @staticmethod
        def getBook(disType):
                dis = None
                if disType == 'EastCoast':
                        dis = EastCoastDistributor()
                elif disType == 'WestCoast':
                        dis = WestCoastDistributor()
                elif disType == 'MidWest':
                        dis = MidWestCoastDistributor()

                dis.ship_book()


# main can be in a separate Client class.
if __name__ == "__main__":
        BookDistributorManager.getBook('WestCoast')
```

What's wrong with this design?

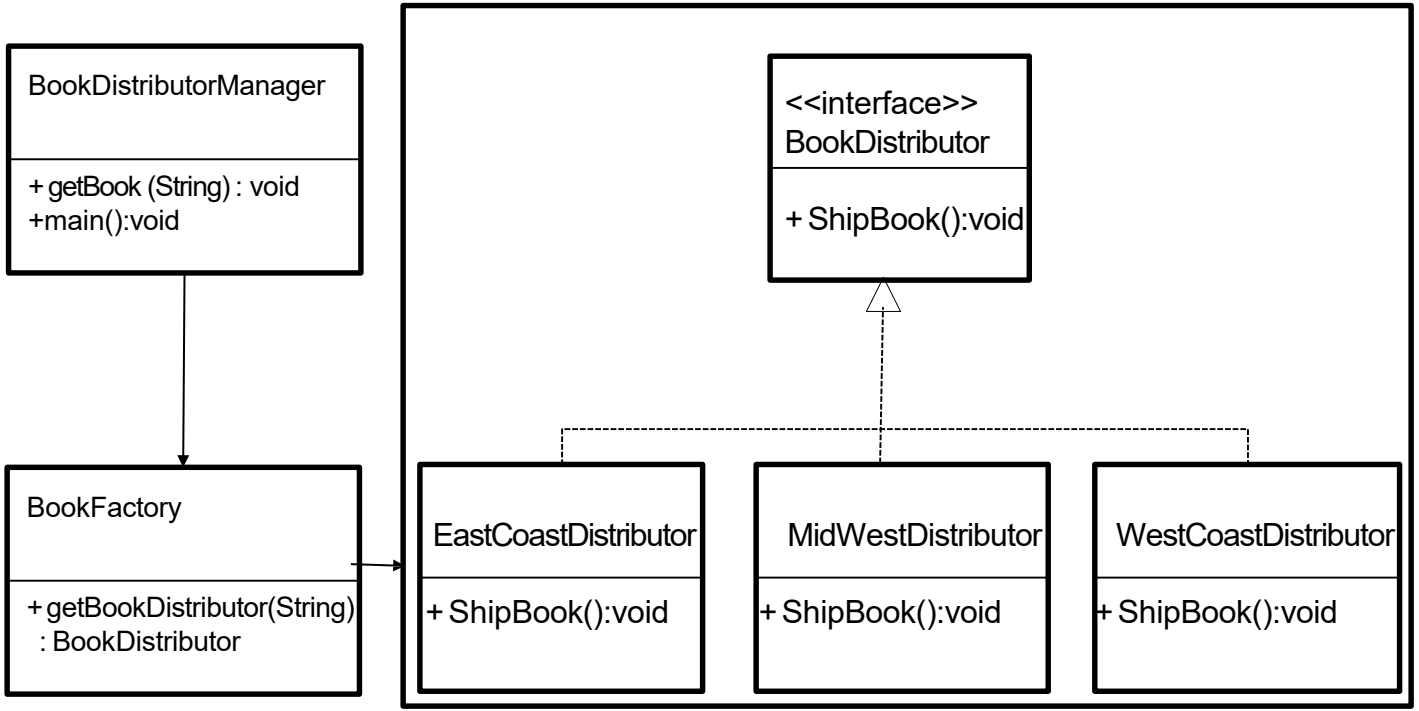
Book shipped from West Coast

Factory Method Example

- Client uses lots of object instantiations
- What happens when new type of distributors will be added in the near future? Client application codes need changed
- Client application has to be aware of all the concrete products (EastCoastDistributor, MidWestDistributor and WestCoastDistributor) and their instantiation logic
- This methodology often introduces a great deal of the inflexibility in the system if the requirements change



Factory Method Example contd.



- Create a Factory to generate object of concrete class based on given information.

```
class BookFactory():

        #getBookDistributor method to get object of type BookDistributor
        @staticmethod
        def getBookDistributor(disType):
                if disType == 'EastCoast’:
                        return EastCoastDistributor()
                elif disType == 'WestCoast’:
                        return WestCoastDistributor()
                elif disType == 'MidWest’:
                        return MidWestCoastDistributor()
                else:
                        return None
```

Use the Factory to get object of concrete class by passing an information such as type.

```
class BookDistributorManager:
        @staticmethod
        def getBook(disType):
                dis = BookFactory.getBookDistributor(disType)
        dis.ship_book()

# main can be in a separate Client class.
if __name__ == "__main__":
        BookDistributorManager.getBook('EastCoast')
```

Book shipped from East Coast

Self study: https://www.geeksforgeeks.org/factory-method-python-design-patterns/?ref=lbp

Factory Pattern

UWE Bristol University of the West of England

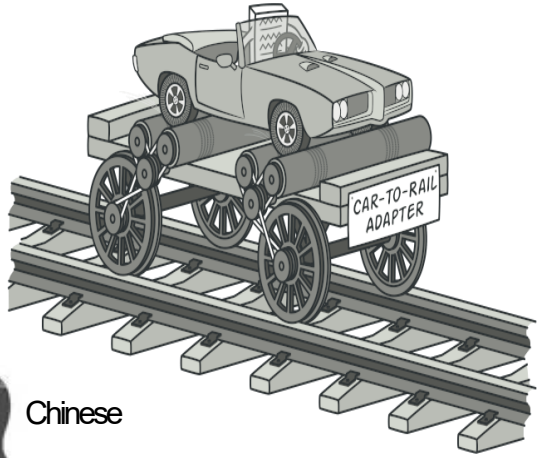
Homework

Do you see the need for Factory Pattern in your group project?

Come prepared for your next practical session

Adapter Pattern

Match interfaces of different classes, coverts an interface to another



Adapter

Japanese

Chinese

Adapter Pattern

UWE Bristol University of the West of England

- Adapter pattern works as a bridge between two incompatible interfaces.
- This pattern involves a single class which is responsible for communication between two independent or incompatible interfaces.
- **Intent**
  - Convert the interface of a class into another interface clients expect
  - Adapter lets classes work together, that could not otherwise because of incompatible interfaces
- **Problem**
  - An "off the shelf" component offers compelling functionality that you would like to reuse, but its "view of the world" is not compatible with the philosophy and architecture of the system currently being developed
- **Discussion**
  - Use it when you need a way to *create a new interface for an object that does the right stuff but has the wrong interface*

Alan Shalloway

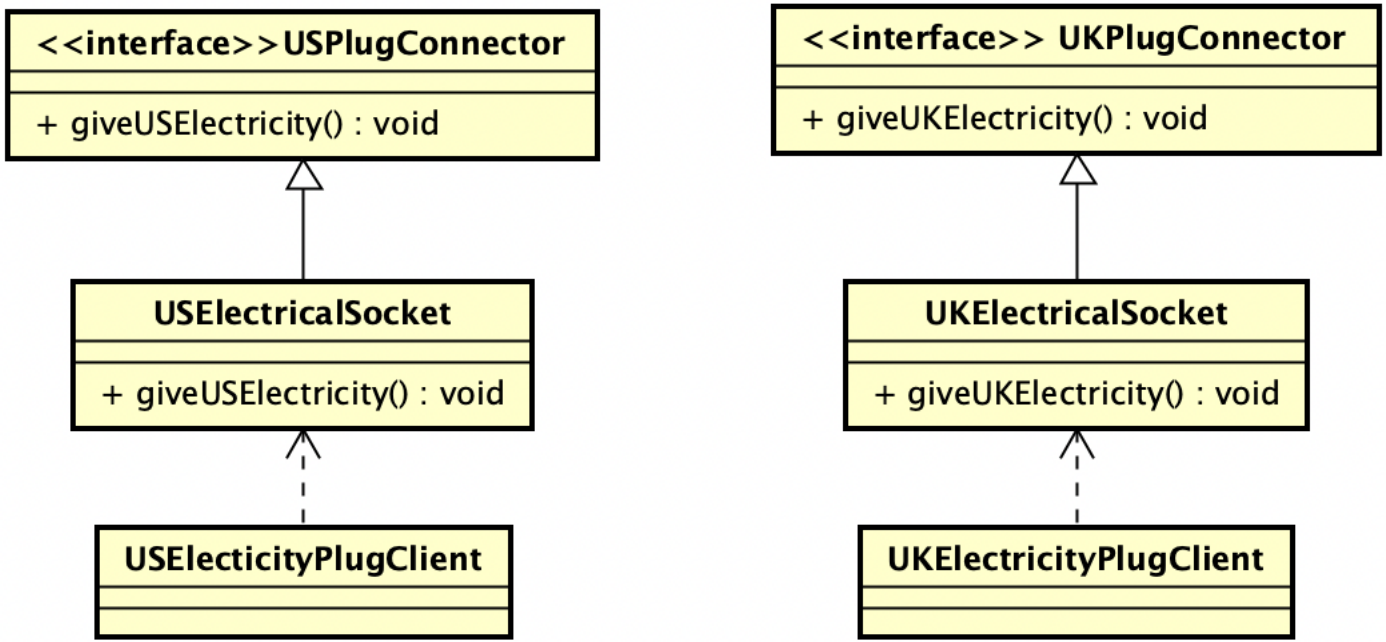
Adapter Pattern

American wall socket

Adapter

UK standard plug

Adapter Pattern



Simple scenarios

US and UK Plug and Socket

```
from abc import ABCMeta, abstractmethod

# the US connector interface
class USPlugConnector(metaclass = ABCMeta):
        @abstractmethod
        def giveUSElectricity():
                pass

# concrete US product/socket
class USElectricalSocket(USPlugConnector):
        def giveUSElectricity(self):
                print('Get US Electiricity')

# the UK connector interface
class UKPlugConnector(metaclass = ABCMeta):
        def giveUKElectricity(self):
                pass

# concrete UK product/socket
class UKElectricalSocket(UKPlugConnector):
        def giveUKElectricity(self):
                print('Get UK Electricity')
```

```
#Testing
class PatternTest:
        usSocket = USElectricalSocket()
        usSocket.giveUSElectricity()

        ukSocket = UKElectricalSocket()
        ukSocket.giveUKElectricity()


# main can be in a separate Client class.
if __name__ == "__main__":
        PatternTest()
```

Output:

Get US Electricity
Get UK Electricity

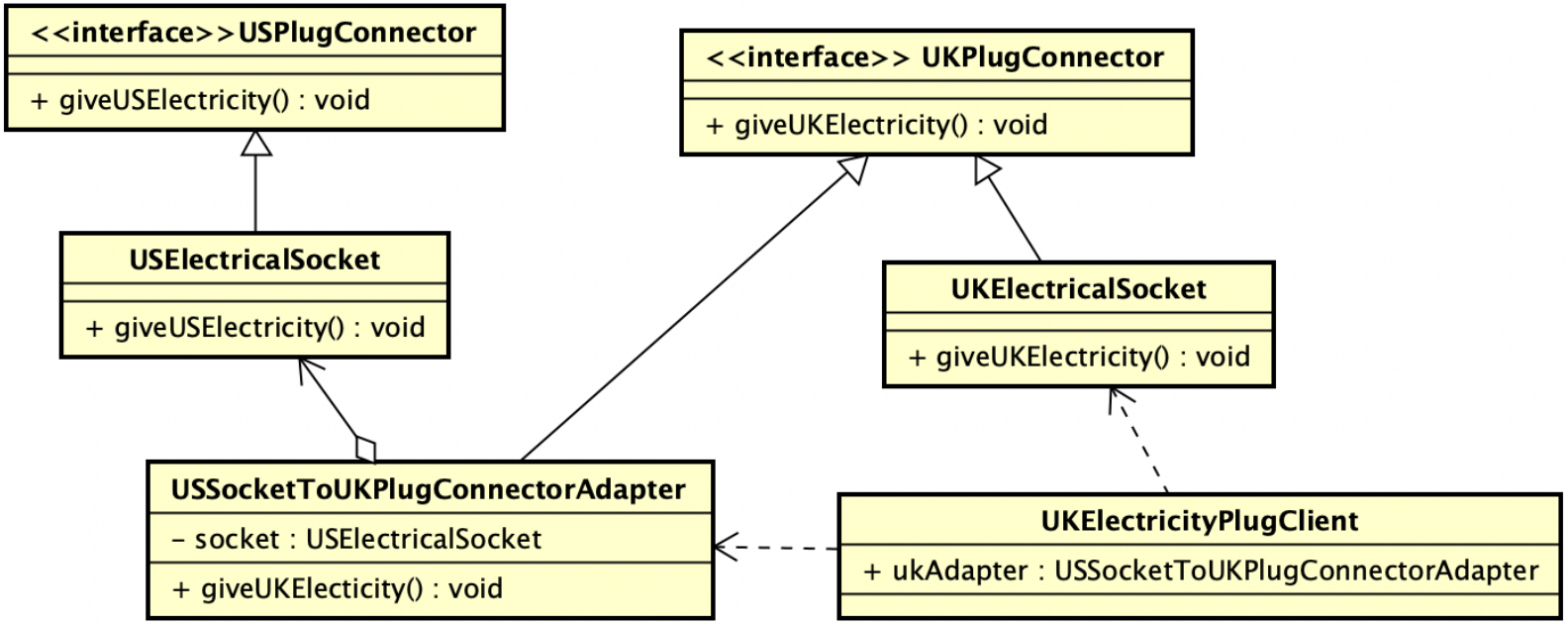
Converts request to be compatible
(Adapter)

Original form of request (Client)

Incompatible (Adaptee)

Client requires service from Adaptee
which is incompatible and cannot interact directly

Adapter Pattern



USToUKPlugConnectorAdapter

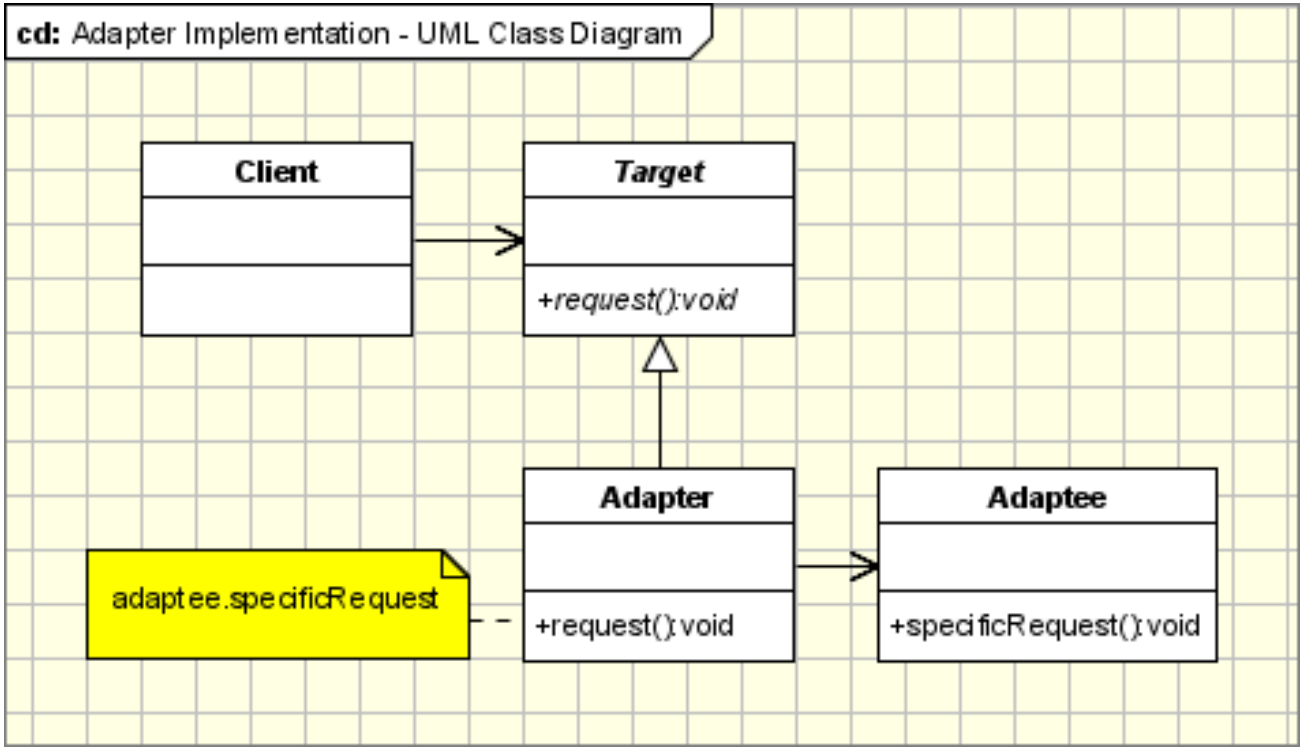
```
#Adapter: adapts the adaptee USSocket so the client UKPlugConnect may use it
# Adapter implements the target i.e. UKPlugConnector and adaptee i.e. USAplugconnector
# serves the delegated request

class USSocketToUKPlugConnectorAdapter(UKPlugConnector):
        __socket = None #private adaptee delegate (uselectricalsocket)

        #adapter implements target
        def USSocketToUKPlugConnectorAdapter(self, uselectricalsocket):
                self.__socket = uselectricalsocket #Adaptee()

        #request - override
        def giveUKElectricity(self):
                self.__socket.giveUSElectricity() #Adpatee.delegatedRequest()
```

USToUKPlugConnectorAdapter

```
#Testing
class PatternTest:
        ukAdapter = USSocketToUKPlugConnectorAdapter()
        ukAdapter.USSocketToUKPlugConnectorAdapter(USElectricalSocket())
        ukAdapter.giveUKElectricity()


# main can be in a separate Client class.
if __name__ == "__main__":
        PatternTest()
```

Output

Get US Electricity


cd: Adapter Implementation - UML Class Diagram
Client
Target
+request():void
Adapter
+request():void
Adaptee
+specificRequest():void
adaptee.specificRequest

Adapter Pattern

UWE Bristol University of the West of England

Homework

Do you see the need for Adapter Pattern in your group project?

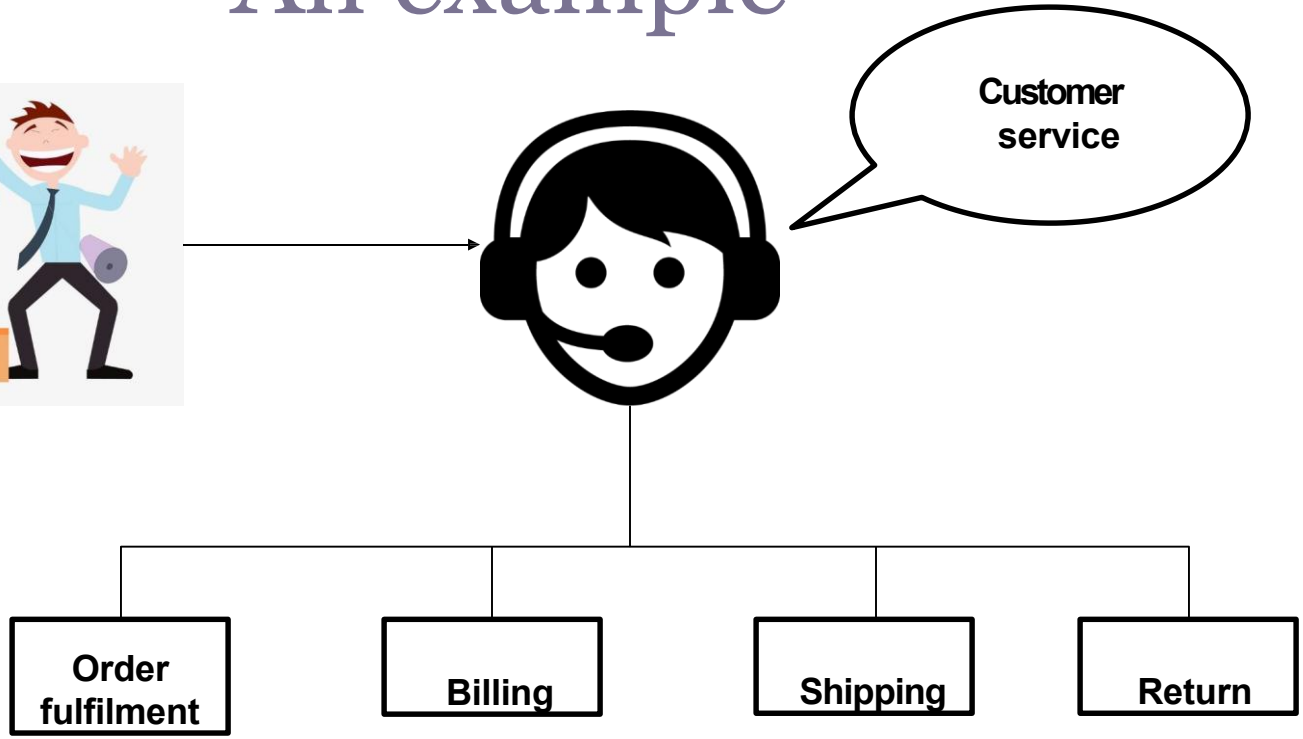
Come prepared for your next practical session

Facade Pattern

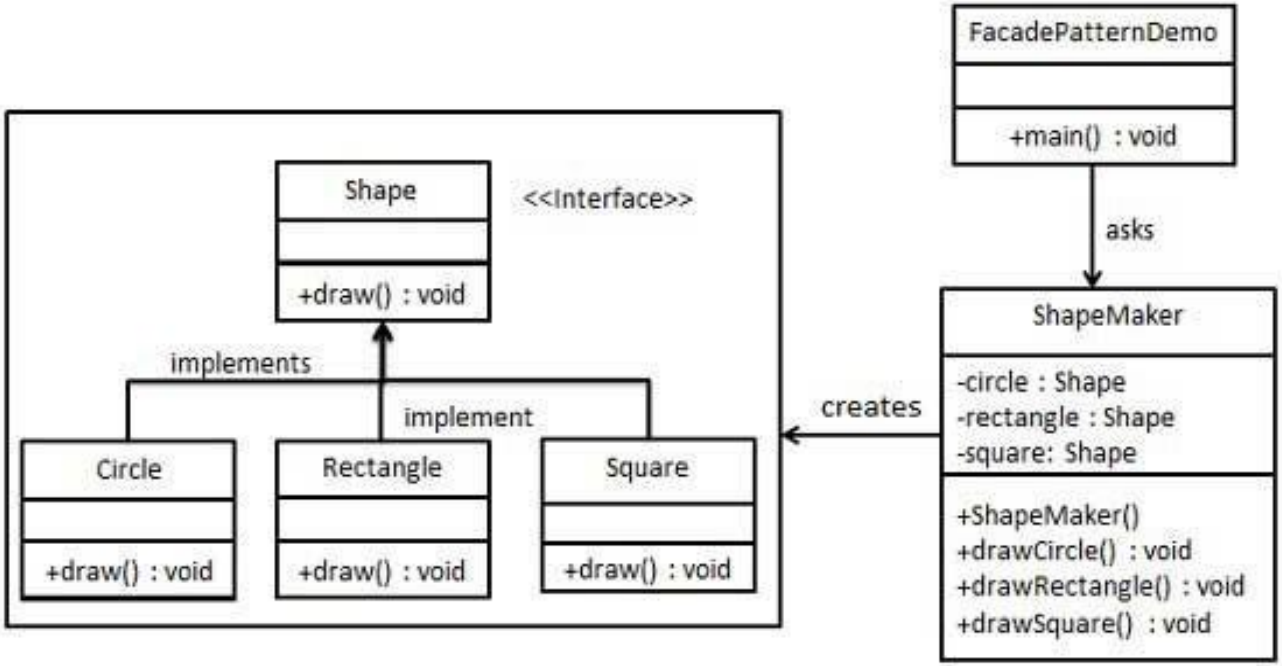
UWE Bristol University of the West of England

- A single class that represents an entire subsystem
- Facade pattern hides the complexities of the system and provides an interface to the client using which the client can access the system
- A facade pattern defines a higher-level interface that makes the subsystem easier to use

An example


Customer service
Order fulfilment
Billing
Shipping
Return

Another Example



```
from abc import ABCMeta, abstractmethod

class Shape(metaclass = ABCMeta):
        @abstractmethod
        def draw():
                pass

class Rectangle(Shape):
        def draw(self):
                print(‘Rectangle::draw()')

class Square(Shape):
        def draw(self):
                print('Square::draw()')

class Circle(Shape):
        def draw(self):
                print('Circle::draw()')
```

Façade class

```
class ShapeMaker():
        __circle = None
        __rectangle = None
        __square = None

        def __init__(self):
                self.__circle = Circle()
                self.__rectangle = Rectangle()
                self.__square = Square()

        def drawCircle(self):
                self.__circle.draw()

        def drawRectangle(self):
                self.__rectangle.draw()

        def drawSquare(self):
                self.__square.draw()
```

Client code

```
#Pattern Testing
class FacadePatternDemo():
        shapeMaker = ShapeMaker()
        shapeMaker.drawCircle()
        shapeMaker.drawRectangle()
        shapeMaker.drawSquare()


# main can be in a separate Client class.
if __name__ == "__main__":
        FacadePatternDemo()
```

Circle::draw()
Rectangle::draw()
Square::draw()

What's the difference between Factory method design pattern and Façade design pattern?

Façade Pattern

UWE Bristol University of the West of England

Homework

Do you see the need for Facade Pattern in your group project?

Come prepared for your next practical session

Observer Pattern

UWE Bristol University of the West of England

- Is a behavioural pattern
  - Behavioural patterns provide solution for the better interaction between objects and how to provide loose coupling and flexibility to extend easily.
  - Better interaction, but low coupling

- Observer pattern:
  - Defines a "one-to-many" dependency between objects so that when one object changes state, all its dependents are notified and updated automatically

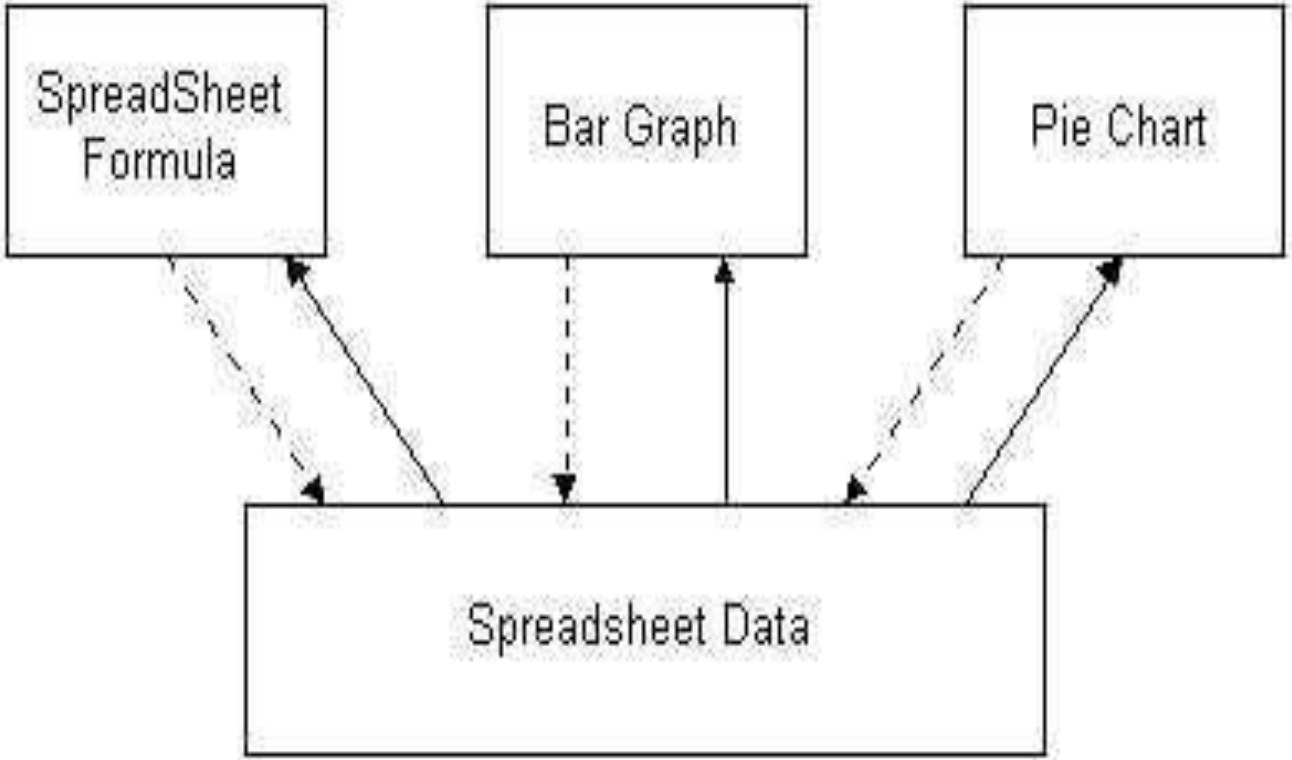
Observer Pattern

- Two parts to the Observer Pattern
  - Subject
  - Observer

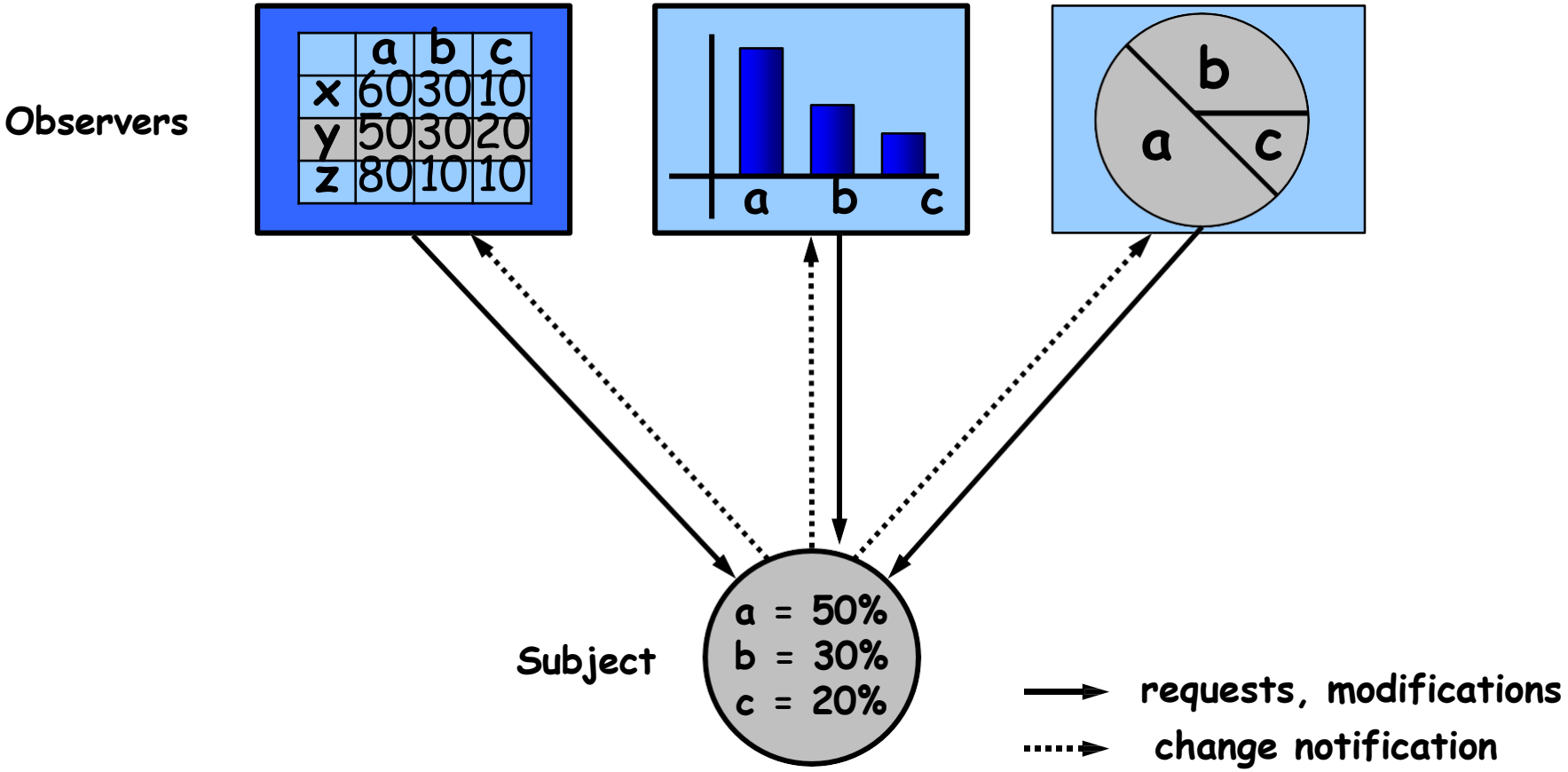
- Provides loosely coupled design between objects that interact this makes the design flexible to accommodate changing requirements
  - There is no need to modify Subject to add or remove observers
  - Subject only knows that observer implements Observer interface – nothing more
  - Subject and Observer classes can be reused independently of each other

Self study: https://www.geeksforgeeks.org/observer-pattern-set-1-introduction/?ref=lbp

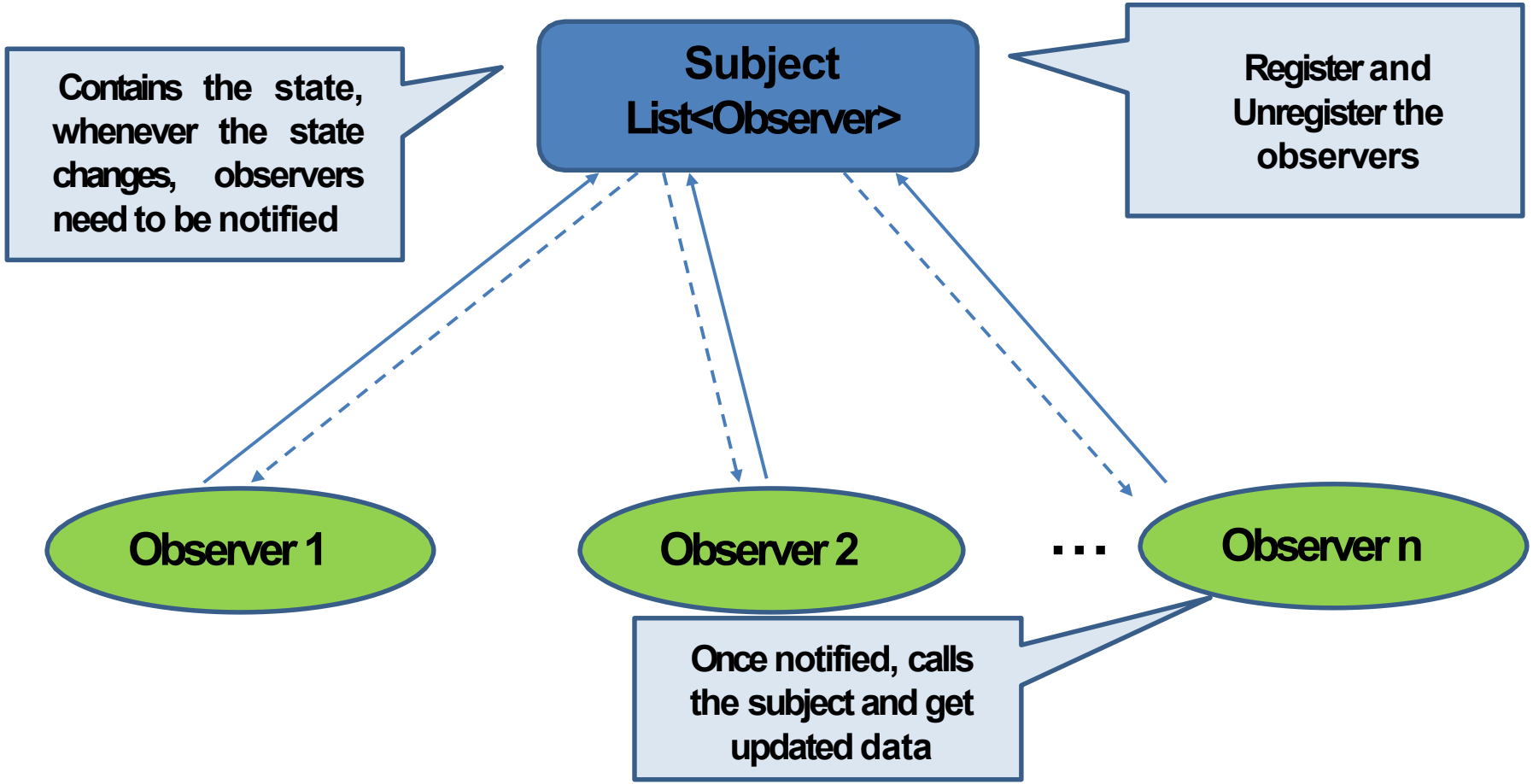
Observer Pattern – Example



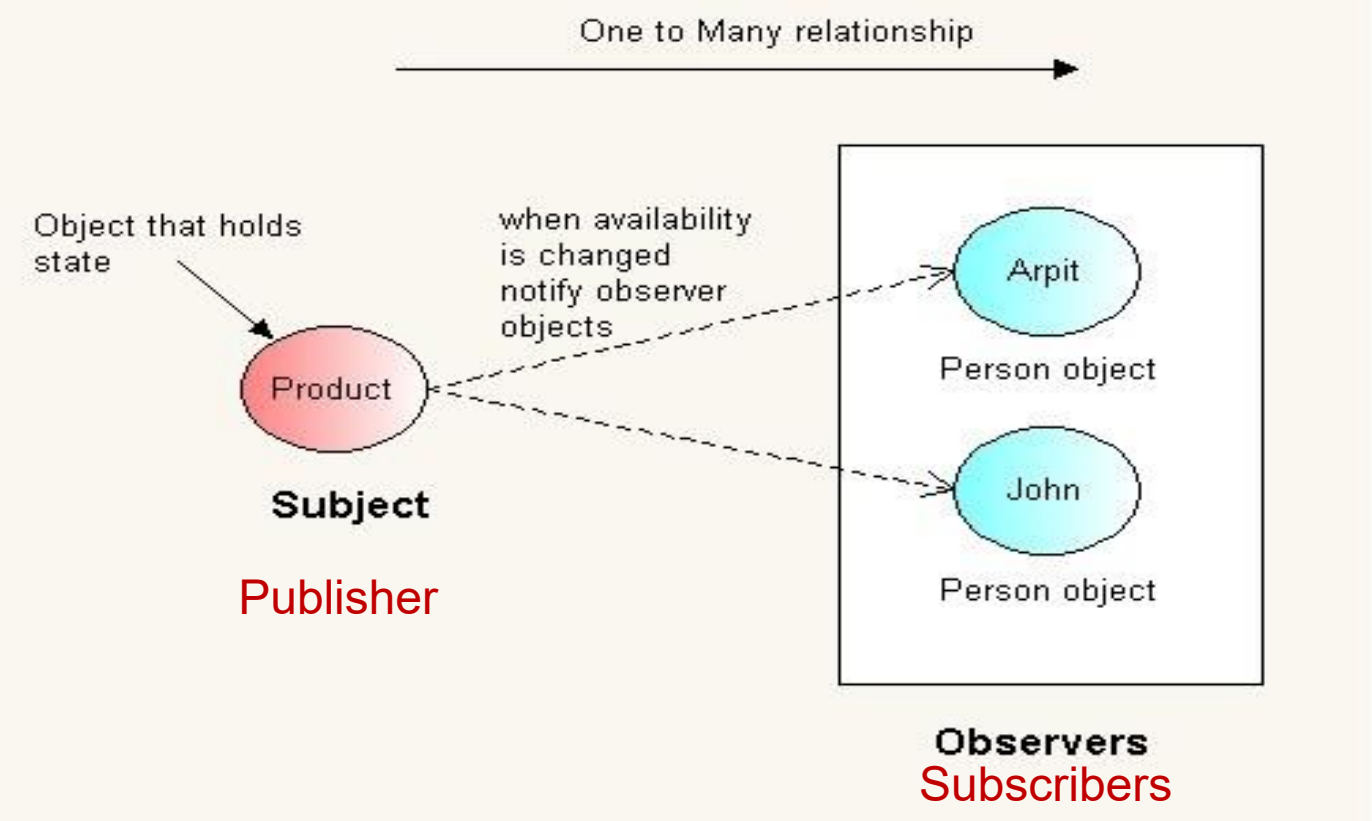
Observer Pattern



Observer Pattern - Structure

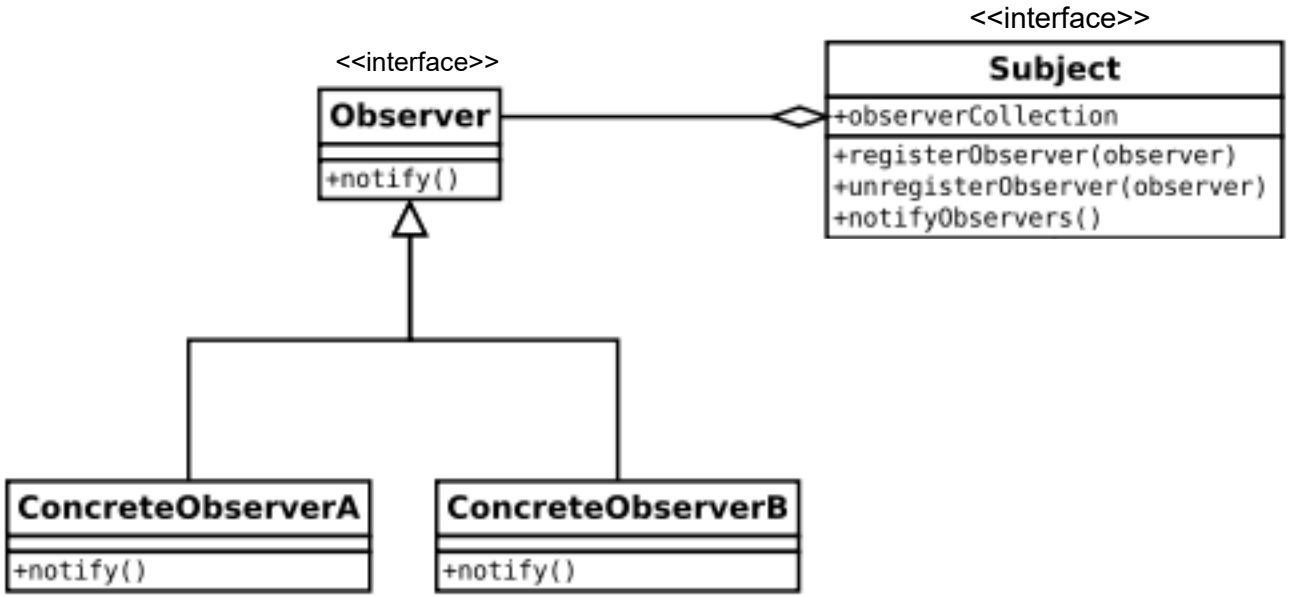


Real example systems...



Source: https://java2blog.com/observer-design-pattern-in-java/

Observer Pattern



notify() method in observer interface defines the action to be taken when the subject provides it data.
observerCollection is registered or subscribed observers

Source: https://www.geeksforgeeks.org/observer-pattern-set-1-introduction/?ref=lbp

Subject

```
class Subject:
        def __init__(self): #Represents what is being observed
                self._observers = [] #create an empty observer list

        def notify(self, modifier = None): #Alert the observers

                for observer in self._observers:
                        if modifier != observer:
                                observer.notify(self)

        def attach(self, observer): #If the observer is not in the list,then append it
                if observer not in self._observers:
                        self._observers.append(observer)

        def detach(self, observer):
                try:
                        self._observers.remove(observer) #Remove observer from the list
                except ValueError:
                        pass
```

Adapted from: https://www.geeksforgeeks.org/observer-method-python-design-patterns/

Subject/Object

```
class Product(Subject):
        """monitor the object/subject"""
        def __init__(self, name =’’):
                Subject.__init__(self)
                self.name = name
                self._quantity = 0

        @property
        def _product(self):
                return self._quantity

        @_product.setter
        def quantity(self, value):
                self._quantity = value
                self.notify()
```

Observer

```
class Observer():
        @abstractmethod
        def notify(self, subject):
                pass

class Customer(Observer):
        #Here notify() can implement action to be taken when notified
        def notify(self, subject):
                print('Customer: Subject/Product {} has number of items in stock
                {}'.format(subject.name, subject._quantity))

class Supplier(Observer):
        #Here notify() can implement action to be taken when notified
        def notify(self, subject):
                print('Supplier: Subject/Product {} has number of items in stock
        {}'.format(subject.name, subject._quantity))

class StockManager(Observer):
        #Here notify() can implement action to be taken when notified
        def notify(self, subject):
                print('Stock Manager: Subject/Product {} has number of items in stock
        {}'.format(subject.name, subject._quantity))
```

Client/Demo

```
if __name__ == "__main__":
        obj1 = Product('MacBook Pro 16inch’)        #product 1
        obj2 = Product('HP Thinkpad’)               #product 2
        #Create couple of suppliers for each product
        s_john = Supplier()
        s_jim = Supplier()
        sm_singh = StockManager() #Create one stockmanager

        c_elias = Customer()                        #Create few customers
        c_zaheer = Customer()
        c_barkha = Customer()
        c_james = Customer()

        #attach two customers and one supplier and stockmanager to product 1
        obj1.attach(c_zaheer)
        obj1.attach(c_james)
        obj1.attach(s_jim)
        obj1.attach(sm_singh)

        #attach two customers and one supplier and stockmanager to product 2
        obj2.attach(c_barkha)
        obj2.attach(c_elias)
        obj2.attach(s_john)
        obj2.attach(sm_singh)

        # The following action of setting data values will notify observers
        obj1.quantity = 10
        obj2.quantity = 20
```

Client/Demo

```
Customer: Subject/Product MacBook Pro 16inch has number of items in stock 10
Customer: Subject/Product MacBook Pro 16inch has number of items in stock 10
Supplier: Subject/Product MacBook Pro 16inch has number of items in stock 10
Stock Manager: Subject/Product MacBook Pro 16inch has number of items in stock 10
Customer: Subject/Product HP Thinkpad has number of items in stock 20
Customer: Subject/Product HP Thinkpad has number of items in stock 20
Supplier: Subject/Product HP Thinkpad has number of items in stock 20
Stock Manager: Subject/Product HP Thinkpad has number of items in stock 20
```

With the above code if we set the quantity to 0 (e.g., `obj1.quantity = 0`), it will still notify to all observers

What should we do to avoid spamming observers e.g., when quantity is 0 then it should not send notification?

Subject/Object

```
class Product(Subject):
        """monitor the object/subject"""
        def __init__(self, name =''):
                Subject.__init__(self)
                self.name = name
                self._quantity = 0

        @property
        def _product(self):
                return self._quantity

        @_product.setter
        def quantity(self, value):
                self._quantity = value
                if self._quantity > 0:
                        self.notify()
```

Observer Pattern

UWE Bristol University of the West of England

Homework

Do you see the need for Observer Pattern in your group project?

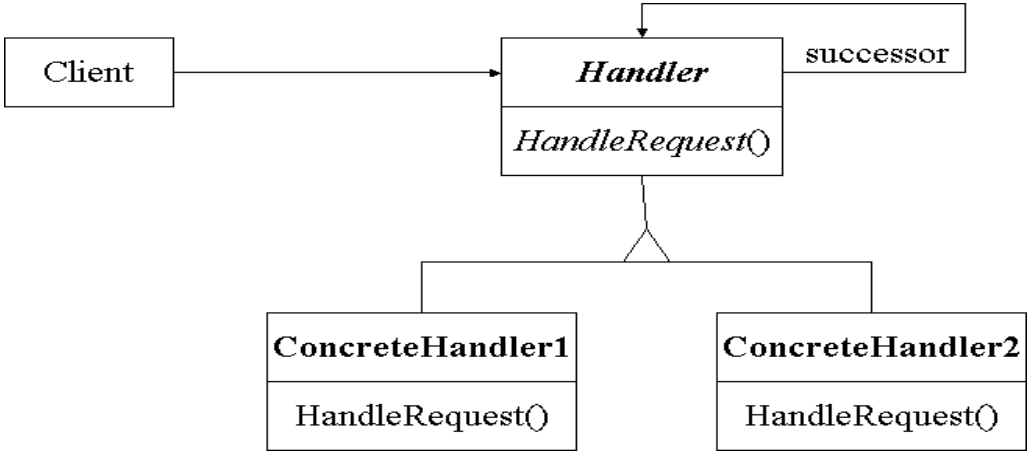
Come prepared for your next practical session

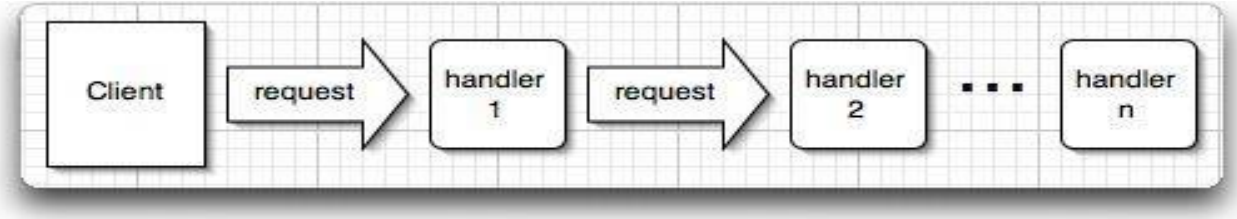
Chain of Responsibility Pattern

- It is used where a request from client is passed to a chain of objects to process the request
- Problem
  - When more than one objects may handle a request and the actual handler is not known in advance
  - When requests follow a "handle or forward" model - that is, some requests can be handled where they are generated while others must be forwarded to another object to be handled
- Consequences
  - Reduced coupling between the sender of a request and the receiver - the sender and receiver have no explicit knowledge of each other
  - Receipt is not guaranteed - a request could fall off the end of the chain without being handled

Chain of Responsibility Pattern contd.

Structure





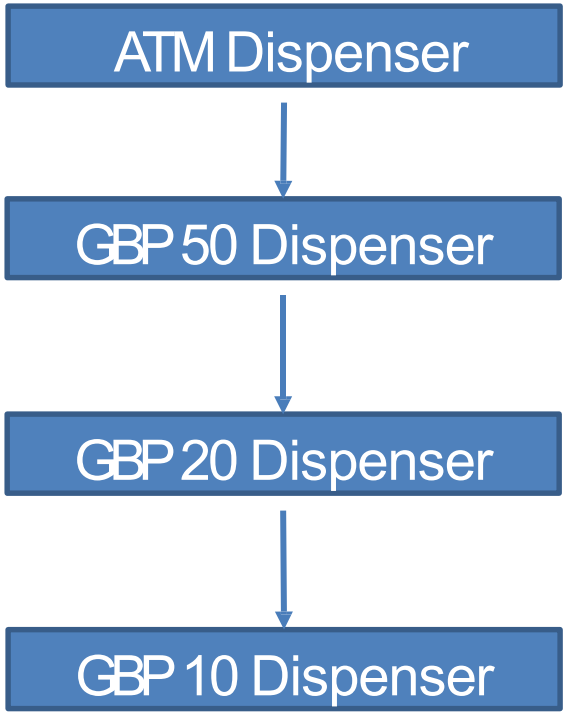
Example

- ATM Dispense machine.

- The user enters the amount to be dispensed and the machine dispense amount in terms of defined currency notes such as 50£, 20£, 10£ etc.

Example contd.

Enter amount to dispense in multiple of 10



Example contd.

```
"""create a class Currency that will store
the amount to dispense and used by the chain
implementations."""

class Currency:
        __amount = 0

        def __init__(self, amount):
                self.__amount = amount

        def getAmount(self):
                return self.__amount
```

```
"""The base interface should have a method to
define the next processor in the chain and the
method that will process the request."""

# interface DispenseChain
class DispenseChain(metaclass = ABCMeta):
        @abstractmethod
        def setNextChain(nextDispenseChain):
                pass

        @abstractmethod
        def dispense(currency):
                pass
```

```
"""We need to create different
processor classes that will implement
the DispenseChain interface and provide
implementation of dispense methods.

Since we are developing our system to work
with three types of currency notes- 50£,
20£ and 10£, we will create three concrete
implementations.
"""
```

```
class GBP50Dispenser(DispenseChain):
        __chain = None

        #override
        def setNextChain(self, nextDispenseChain):
                self.__chain = nextDispenseChain

        #override
        def dispense(self, currency):
                amount = currency.getAmount()
                if amount >= 50:
                        num = amount // 50
                        remainder = amount % 50
                        print ("Dispensing " + str(num) + " 50 £ notes")
                        if remainder != 0:
                                self.__chain.dispense(Currency(remainder))
                else:
                        self.__chain.dispense(currency)
```

```
class GBP20Dispenser(DispenseChain):
        __chain = None

        #override
        def setNextChain(self, nextDispenseChain):
                self.__chain = nextDispenseChain

        #override
        def dispense(self, currency):
                amount = currency.getAmount()
                if amount >= 20:
                        num = amount // 20
                        remainder = amount % 20
                        print ("Dispensing " + str(num) + " 20 £ notes")
                        if remainder != 0:
                                self.__chain.dispense(Currency(remainder))
                else:
                        self.__chain.dispense(currency)
```

```
class GBP10Dispenser(DispenseChain):
        __chain = None

        #override
        def setNextChain(self, nextDispenseChain):
                self.__chain = nextDispenseChain

        #override
        def dispense(self, currency):
                amount = currency.getAmount()
                if amount >= 10:
                        num = amount // 10
                        remainder = amount % 10
                        print ("Dispensing " + str(num) + " 10 £ notes")
                        if remainder != 0:
                                self.__chain.dispense(Currency(remainder))
                else:
                        self.__chain.dispense(currency)
```

"""This is a very important step and we should create the chain carefully, otherwise a processor might not be getting any request at all.

For example, in our implementation if we keep the first processor chain as GBP10Dispenser and then GBP20Dispenser, then the request will never be forwarded to the second processor and the chain will become useless."""

```
class ATMDispenseChain:
        __c1 = None

        def __init__(self):
                #initialise the chain
                self.__c1 = GBP50Dispenser()
                self.c2 = GBP20Dispenser()
                self.c3 = GBP10Dispenser()

                #set the chain of responsibility
                self.__c1.setNextChain(self.c2)
                self.c2.setNextChain(self.c3)

        def getFristChain(self):
                return self.__c1
```

Client/Demo

```
# main can be in a separate Client class.
if __name__ == "__main__":
        atmDispenser = ATMDispenseChain()

        while True:
                amount = 0
                print('Enter amount to dispense: Enter 0 to exit \n’)
                amount = int(input())
                if amount == 0:
                        exit()
                remainder = amount % 10
                if remainder != 0:
                        print('Amount should be multiple of 10s.’)
                        continue

                atmDispenser.getFristChain().dispense(Currency(amount))
```

Output

```
Enter amount to dispense: Enter 0 to exit

500
Dispensing 10 50 £ notes
Enter amount to dispense: Enter 0 to exit

430
Dispensing 8 50 £ notes
Dispensing 1 20 £ notes
Dispensing 1 10 £ notes
Enter amount to dispense: Enter 0 to exit

310
Dispensing 6 50 £ notes
Dispensing 1 10 £ notes
Enter amount to dispense: Enter 0 to exit

50
Dispensing 1 50 £ notes
Enter amount to dispense: Enter 0 to exit

40
Dispensing 2 20 £ notes
Enter amount to dispense: Enter 0 to exit

10
Dispensing 1 10 £ notes
Enter amount to dispense: Enter 0 to exit

5
Amount should be multiple of 10s.
Enter amount to dispense: Enter 0 to exit

0
```

Important points

- Client doesn't know which part of the chain will be processing the request and it will send the request to the first object in the chain. For example, in our program ATMDispenseChain is unaware of who is processing the request to dispense the entered amount.

- Each object in the chain will have it's own implementation to process the request, either full or partial or to send it to the next object in the chain.

- Every object in the chain should have reference to the next object in chain to forward the request to

Observation…

- Why cannot we merge BGP50Dispense, BGP20Dispense, and BGP10Dispense classes into a single class? (code put together)
  - Which SOLID principle(s) will be violated?

- Do you see any problem in the ATMDispenseChain implementation?
  - How can we improve this code?

Chain of responsibility Pattern

UWE Bristol University of the West of England

Homework

Do you see the need for Chain of Responsibility Pattern in your group project?

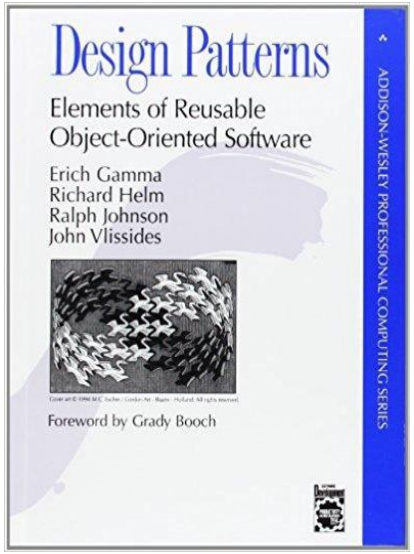
Come prepared for your next practical session

Summary...

- There are several design patterns and we just covered a few
- Design patterns help software developers to apply solutions to commonly faced software design problems
- They provide standard terminology and are specific to particular scenario
- Learning these patterns helps unexperienced developers to learn software design in an easy and faster way
- Creational patterns are about class instantiation
- Structural patterns are about class and object composition
- Behavioural patterns are about better interaction and loose coupling

Suggested Readings

- Online (Python specific):
  - https://www.geeksforgeeks.org/python-design-patterns/
  - https://python-patterns.guide/
  - https://refactoring.guru/design-patterns/python
  - https://www.tutorialspoint.com/design_pattern/index.htm
  - https://www.journaldev.com/1827/java-design-patterns-example- tutorial#observer-pattern



- Head First Design Patterns: Building Extensible and maintainable OO Software by Freeman, Eric, Robson, Elisabeth, 2nd Edition, Wiley